DP Computer Science: A 4.3.6 / 4.3.7 Reinforcement Learning & Genetic Algorithms

1. Core Concepts & Learning Objectives

Learning Intentions

By the end of this lesson, students will be able to:

- Describe the **agent-environment interaction loop** in Reinforcement Learning (RL)
- Explain the core concepts of RL, including **cumulative reward** and the **exploration vs. exploitation** trade-off
- Describe the core concepts of **Genetic Algorithms (GAs)** , including **population, fitness, selection, crossover, and mutation**
- **Differentiate** between RL and GAs as problem-solving approaches
- Describe a **real-world application** for both RL (e.g., robotics) and GAs (e.g., optimisation problems like the travelling salesperson)

Key Vocabulary

Term	Definition
Reinforcement Learning (RL)	A learning paradigm where an agent learns to make decisions by interacting with an environment and receiving rewards
Agent	The decision-maker in an RL system
Environment	The world the agent interacts with
Action	A decision or move made by the agent
State	The current situation of the agent in the environment

Term	Definition
Reward	A signal that indicates how good or bad an action was
Policy	The agent's strategy for deciding actions based on states
Exploration	Trying new actions to discover their effects
Exploitation	Using known actions that give high rewards
Genetic Algorithm (GA)	An optimisation technique inspired by natural selection and evolution
Population	A set of candidate solutions in a GA
Fitness function	A measure of how good a solution is
Selection	Choosing the best solutions to reproduce
Crossover	Combining two solutions to create a new one
Mutation	Randomly changing a solution to introduce variation

Approaches to Learning (ATL) Elements

Skill	Description
Thinking (Conceptual Understanding & Comparative Analysis)	Understanding two distinct learning paradigms and comparing their strengths, weaknesses, and use cases
Communication	Using analogies to explain abstract, bio-inspired or interaction-based concepts

2. Introduction: Two Different Learning Paradigms

The Hook: How Do We Learn Without Data?

```text
┌──────────────────────────────────────────────────────────────────┐
│              TWO LEARNING PARADIGMS                              │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│    ┌─────────────────────────────────────────────────────────┐   │
│    │              HOW DOES A BABY LEARN TO WALK?             │   │
│    │                                                         │   │
│    │    • Trial and error                                    │   │
│    │    • Falls down → learns not to do that                 │   │
│    │    • Takes a step → gets rewarded with progress         │   │
│    │    • Learns by interacting with the environment         │   │
│    │                                                         │   │
│    │    This is REINFORCEMENT LEARNING!                      │   │
│    └─────────────────────────────────────────────────────────┘   │
│                                                                  │
│    ┌─────────────────────────────────────────────────────────┐   │
│    │              HOW DOES NATURE 'DESIGN' A CHEETAH?        │   │
│    │                                                         │   │
│    │    • Evolution over many generations                    │   │
│    │    • Cheetahs that run faster survive                   │   │
│    │    • They reproduce, passing on good traits             │   │
│    │    • Over time, cheetahs become faster                  │   │
│    │                                                         │   │
│    │    This is a GENETIC ALGORITHM!                         │   │
│    └─────────────────────────────────────────────────────────┘   │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

---

3. Reinforcement Learning (A4.3.6)

What is Reinforcement Learning?

> Reinforcement Learning is a learning paradigm where an agent learns to make decisions by interacting with an environment and receiving rewards (or punishments). The goal is to maximise cumulative reward over time.

The Agent-Environment Loop

```text
┌─────────────────────────────────────────────────────────────────┐
│              AGENT-ENVIRONMENT LOOP                            │
├─────────────────────────────────────────────────────────────────┤
│                                                                 │
│                     ┌──────────────────────────────────────┐   │
│                     │             AGENT                    │   │
│                     │        (The Decision Maker)          │   │
│                     └──────────────┬───────────────────────┘   │
│                                    │                            │
│                             Action │                            │
│                                    │                            │
│                                    ▼                            │
│                     ┌──────────────────────────────────────┐   │
│                     │          ENVIRONMENT                 │   │
│                     │      (The World the Agent            │   │
│                     │        Interacts With)               │   │
│                     └──────────────┬───────────────────────┘   │
│                                    │                            │
│                     State + Reward │                            │
│                                    │                            │
│                                    ▼                            │
│                     (Loop Continues)                           │
│                                                                 │
│    Example: Robot Vacuum Cleaner                               │
│    • Agent: The vacuum                                         │
│    • Environment: The room                                     │
│    • Action: Move forward, turn left, turn right, vacuum      │
│    • State: Current position, which areas are clean           │
│    • Reward: +1 for cleaning a spot, -1 for hitting a wall   │
│                                                                 │
└─────────────────────────────────────────────────────────────────┘
```

Core Concepts of RL

Concept	Definition	Example (Robot Vacuum)
Agent	The decision-maker	The vacuum cleaner
Environment	The world the agent interacts with	The room being cleaned
Action	What the agent can do	Move forward, turn left, turn right, vacuum
State	The current situation	Position, which areas are clean
Reward	Feedback signal	+1 for cleaning a spot, -1 for hitting a wall
Policy	The agent's strategy for choosing actions	"If there is dirt in front of me, vacuum; otherwise, move forward"

Exploration vs. Exploitation

> **The Exploration vs. Exploitation trade-off is one of the key challenges in RL: should the agent try new actions (exploration) or use actions it already knows work well (exploitation)?**

```text
┌─────────────────────────────────────────────────────────────────┐
│          EXPLORATION vs. EXPLOITATION                          │
├─────────────────────────────────────────────────────────────────┤
│                                                                 │
│    ┌─────────────────────────────────────────────────────────┐  │
│    │             EXPLORATION                                │  │
│    │                                                         │  │
│    │    • Try new actions to discover their effects         │  │
│    │    • "Let's try this new path; maybe it's faster"     │  │
│    │    • Risk: Might perform poorly                          │  │
│    │    • Benefit: May discover better strategies            │  │
│    └─────────────────────────────────────────────────────────┘  │
│                                                                 │
│    ┌─────────────────────────────────────────────────────────┐  │
│    │             EXPLOITATION                               │  │
│    │                                                         │  │
│    │    • Use known actions that give high rewards          │  │
│    │    • "I know this path works well, so I'll use it"    │  │
```

```
│    │    • Risk: Might miss a better strategy          │  │
│    │    • Benefit: Reliable performance               │  │
│    └──────────────────────────────────────────────────┘  │
│                                                           │
│    Analogy: Going to your favourite restaurant vs.        │  │
│              trying a new restaurant.                   │  │
│                                                           │
└───────────────────────────────────────────────────────────┘
```

Real-World Applications of RL

Application	Description
Robotics	Robots learn to navigate, manipulate objects, and perform tasks
Game Playing	AlphaGo, chess, and other game-playing AI
Autonomous Vehicles	Self-driving cars learn to navigate roads
Healthcare	Personalised treatment plans
Traffic Optimisation	Adaptive traffic light control
Recommendation Systems	Learning user preferences over time

---

4. Genetic Algorithms (A4.3.7)

What is a Genetic Algorithm?

> **A Genetic Algorithm is an optimisation technique inspired by natural selection and evolution. It evolves a population of solutions over generations to find the best solution to a problem.**

The Travelling Salesperson Problem (TSP)

> **The TSP is a classic optimisation problem: given a list of cities, find the shortest possible route that visits each city exactly once and returns to the starting point.**

How Genetic Algorithms Work

```text
┌─────────────────────────────────────────────────────────────────┐
│           GENETIC ALGORITHM PROCESS                             │
├─────────────────────────────────────────────────────────────────┤
│                                                                 │
│    ┌─────────────────────────────────────────────────────────┐  │
│    │  1. Initialise Population                               │  │
│    │     Create a random set of solutions (routes)           │  │
│    └─────────────────────────────────────────────────────────┘  │
│                               │                                 │
│                               ▼                                 │
│    ┌─────────────────────────────────────────────────────────┐  │
│    │  2. Evaluate Fitness                                    │  │
│    │     Score each solution (e.g., total distance)          │  │
│    └─────────────────────────────────────────────────────────┘  │
│                               │                                 │
│                               ▼                                 │
│    ┌─────────────────────────────────────────────────────────┐  │
│    │  3. Selection                                           │  │
│    │     Choose the best solutions to reproduce              │  │
│    └─────────────────────────────────────────────────────────┘  │
│                               │                                 │
│                               ▼                                 │
│    ┌─────────────────────────────────────────────────────────┐  │
│    │  4. Crossover                                           │  │
│    │     Combine two solutions to create new ones            │  │
│    └─────────────────────────────────────────────────────────┘  │
│                               │                                 │
│                               ▼                                 │
│    ┌─────────────────────────────────────────────────────────┐  │
│    │  5. Mutation                                            │  │
│    │     Make small random changes to solutions              │  │
│    └─────────────────────────────────────────────────────────┘  │
│                               │                                 │
│                               ▼                                 │
│    ┌─────────────────────────────────────────────────────────┐  │
│    │  6. Evaluate New Generation                             │  │
│    │     Repeat from Step 2 until termination condition      │  │
│    └─────────────────────────────────────────────────────────┘  │
│                                                                 │
```

```
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

Core Concepts of GAs

```text

┌──────────────────────────────────────────────────────────────────┐
│              GA CONCEPTS EXPLAINED                              │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│    ┌──────────────────────────────────────────────────────────┐  │
│    │              POPULATION                                 │  │
│    │      A set of candidate solutions                       │  │
│    │      Example: 100 different routes for the TSP        │  │
│    └──────────────────────────────────────────────────────────┘  │
│                                                                  │
│    ┌──────────────────────────────────────────────────────────┐  │
│    │              FITNESS FUNCTION                             │  │
│    │      A measure of how good a solution is                │  │
│    │      Example: Total distance travelled                  │  │
│    └──────────────────────────────────────────────────────────┘  │
│                                                                  │
│    ┌──────────────────────────────────────────────────────────┐  │
│    │              SELECTION                                  │  │
│    │      Choosing the best solutions to reproduce           │  │
│    │      Example: The 10 shortest routes                    │  │
│    └──────────────────────────────────────────────────────────┘  │
│                                                                  │
│    ┌──────────────────────────────────────────────────────────┐  │
│    │              CROSSOVER                                  │  │
│    │      Combining two solutions to create new ones         │  │
│    │      Example: Taking part of one route and part of    │  │
│    │               another                                   │  │
│    └──────────────────────────────────────────────────────────┘  │
│                                                                  │
│    ┌──────────────────────────────────────────────────────────┐  │
│    │              MUTATION                                   │  │
│    │      Making small random changes to solutions           │  │
│    │      Example: Swapping the order of two cities        │  │
│    └──────────────────────────────────────────────────────────┘  │
```

```
│                                                                 │
└─────────────────────────────────────────────────────────────────┘
```

Visual Example: TSP Crossover

```text
┌─────────────────────────────────────────────────────────────────┐
│              CROSSOVER EXAMPLE (TSP)                            │
├─────────────────────────────────────────────────────────────────┤
│                                                                 │
│    Parent 1: A → B → C → D → E → F                              │
│    Parent 2: A → C → F → B → E → D                              │
│                                                                 │
│    ─────────────────────────────────────────────────────────    │
│                                                                 │
│    Crossover Point: After city 3                                │
│                                                                 │
│    Parent 1: A → B → C | D → E → F                              │
│    Parent 2: A → C → F | B → E → D                              │
│                                                                 │
│    Child 1: A → B → C | B → E → D  (duplicates)                 │
│    Child 2: A → C → F | D → E → F  (duplicates)                 │
│                                                                 │
│    After Fixing Duplicates:                                     │
│                                                                 │
│    Child 1: A → B → C → E → D → F                               │
│    Child 2: A → C → F → D → E → B                               │
│                                                                 │
│    The child routes are new solutions!                          │
│                                                                 │
└─────────────────────────────────────────────────────────────────┘
```

Real-World Applications of GAs

Application	Description
Route Optimisation	Finding the shortest path (Travelling Salesperson)

Application	Description
Scheduling	Optimising timetables and work schedules
Engineering Design	Designing lightweight but strong structures
Financial Modelling	Optimising investment portfolios
Game AI	Evolving strategies for games
Drug Discovery	Finding optimal molecular structures

---

5. RL vs. Genetic Algorithms: Comparison

Comparison Table

Feature	Reinforcement Learning	Genetic Algorithms
Inspiration	Human/animal learning (trial and error)	Biological evolution (natural selection)
Learning Process	Agent interacts with environment	Population evolves over generations
Feedback	Rewards (positive or negative)	Fitness score
Goal	Maximise cumulative reward	Find optimal solution
Time Frame	Continuous learning over time	Generations (iterations)
Best For	Sequential decision-making	Optimisation problems
Example	Self-driving car	Shortest route planning
Output	A policy (strategy)	A single best solution

When to Use Which?

```text
┌─────────────────────────────────────────────────────────────────┐
│              RL vs. GA: WHEN TO USE WHICH?                      │
├─────────────────────────────────────────────────────────────────┤
│                                                                 │
│    ┌─────────────────────────────────────────────────────────┐  │
│    │              REINFORCEMENT LEARNING                     │  │
│    │                                                         │  │
│    │    Use when:                                            │  │
│    │    • There is an environment to interact with           │  │
│    │    • Decisions are made sequentially                    │  │
│    │    • The agent learns over time                         │  │
│    │    • Examples: Robotics, game playing, autonomous       │  │
│    │      vehicles                                           │  │
│    └─────────────────────────────────────────────────────────┘  │
│                                                                 │
│    ┌─────────────────────────────────────────────────────────┐  │
│    │              GENETIC ALGORITHM                          │  │
│    │                                                         │  │
│    │    Use when:                                            │  │
│    │    • There is a defined optimisation problem            │  │
│    │    • Solutions can be scored with a fitness function    │  │
│    │    • The solution is static (not continuously changing) │  │
│    │    • Examples: Route optimisation, scheduling,          │  │
│    │      engineering design                                 │  │
│    └─────────────────────────────────────────────────────────┘  │
│                                                                 │
└─────────────────────────────────────────────────────────────────┘
```

---

6. Activity: "RL or GA?"

Scenario 1: Smart Home Heating System

Problem: Designing a heating and cooling system for a smart home that learns the owner's preferences over time to save energy.

Aspect	Analysis
Type	**Reinforcement Learning**
Why?	The system must learn continuously over time by interacting with the environment (the home and user). It needs to adapt to changing patterns and feedback (user comfort).

Scenario 2: Bridge Support Design

Problem: Finding the optimal structural design for a lightweight but strong bridge support.

Aspect	Analysis
Type	**Genetic Algorithm**
Why?	This is a one-time optimisation problem. The goal is to find a single optimal structure. A GA can evolve designs to find the best balance of weight and strength.

Additional Scenarios for Discussion

Scenario	Technique	Why?
Robotic arm learning to pick up objects	RL	Sequential decision-making; learns from interaction
Finding the shortest delivery route	GA	Optimisation problem; static solution
Game AI that improves over time	RL	Learns from interactions with the game
Optimal antenna design	GA	One-time optimisation; design can be scored
Traffic light optimisation	RL	Continuous adaptation to traffic patterns

7. Lesson Sequence

Lesson Plan (60 minutes)

Phase	Time	Activity
Recap & Hook	10 min	Recap unsupervised learning; introduce two learning paradigms (baby learning to walk, evolution)
Lecture: RL	15 min	Agent-environment loop; core concepts; exploration vs. exploitation
Lecture: GA	15 min	Travelling Salesperson Problem; population, fitness, selection, crossover, mutation
Conceptual Activity	15 min	"RL or GA?" — students decide which technique is better for given scenarios
Consolidation	5 min	Review answers; compare RL and GA; preview next lesson

---

Discussion Questions

Question	Purpose
"What is the agent-environment loop in RL?"	Understand RL framework
"What is the exploration vs. exploitation trade-off?"	Understand RL challenge
"How does a genetic algorithm work?"	Understand GA process
"When would you use RL instead of a GA?"	Compare approaches
"Give an example of a problem suited to a GA."	Apply knowledge

---

8. Assessment Activities

Assessment Type	Description
Class Participation	Observe engagement and contributions
"RL or GA?" Activity	Assess ability to differentiate techniques
Exit Ticket	Gauge understanding and identify areas needing clarification

Sample Exit Ticket Questions:

1. What is the agent-environment loop in RL?
2. What is the difference between exploration and exploitation?
3. Name the five core concepts of a genetic algorithm.
4. When would you use a genetic algorithm instead of reinforcement learning?

---

9. Differentiation

Level	Strategy
Support	Use clear analogies and visual examples; step through concepts carefully
Challenge	Research real-world implementations (e.g., AlphaGo for RL; NASA's antenna design for GAs)

---

10. Resources

- **Presentation** (Slide Deck)
- **eBook** (A4.3.6 & A4.3.7 with examples)
- **Worksheet:** RL vs. GA scenarios
- **Worksheet Answers**
- **Quizlet** (A4.3.6/A4.3.7 vocabulary flashcards)

11. Summary: Key Takeaways

Concept	Key Point
Reinforcement Learning	Learns by interacting with an environment and receiving rewards
Agent-Environment Loop	Action → Environment → State + Reward → Action
Exploration	Trying new actions to discover their effects
Exploitation	Using known actions that give high rewards
Genetic Algorithm	Optimisation technique inspired by evolution
Population	Set of candidate solutions
Fitness	Measure of how good a solution is
Selection	Choosing the best solutions to reproduce
Crossover	Combining two solutions
Mutation	Randomly changing a solution

```text
┌─────────────────────────────────────────────────────────────────┐
│                    KEY REMINDERS                                │
├─────────────────────────────────────────────────────────────────┤
│                                                                 │
│    ✅ RL = Learning by interacting with an environment          │
│    ✅ Agent → Action → Environment → State + Reward            │
│    ✅ Exploration = try new things                              │
│    ✅ Exploitation = use what works                             │
│    ✅ GA = Evolution-inspired optimisation                     │
│    ✅ Population = set of solutions                             │
│    ✅ Fitness = how good a solution is                          │
│    ✅ Selection = choose the best                               │
│    ✅ Crossover = combine solutions                             │
```

```
│     ✅ Mutation = make random changes                          │
│                                                                │
└────────────────────────────────────────────────────────────────┘
```

> "*Reinforcement Learning and Genetic Algorithms represent two powerful ways to solve problems—one through learning from experience, the other through simulated evolution.*"